

Challenge: Multi-Agent Collaboration System

Description

Most people's first "AI project" is a single LLM call with a clever prompt. This challenge pushes you one level up: build a system where multiple agents, each with their own role, memory, and responsibilities, work together to complete a task neither could do as well alone.

Concretely, you'll design a small team of agents (2-3) that collaborate on a task with a natural division of labor. The classic example is a **researcher** → **writer** → **critic** pipeline: one agent gathers information, another turns it into a draft, and a third reviews and pushes back on it, possibly sending it back for revision. But the task itself is up to you (see Creative Freedom below).

The point isn't just "call the LLM three times." It's to build a system where agents have distinct roles, can pass structured information between each other, and where the final output is genuinely better because multiple perspectives were involved. This is exactly the kind of orchestration that shows up in real production AI systems, and frameworks like LangGraph, CrewAI, and AutoGen exist specifically to make this easier.

Steps to Get There

You're free to approach this however makes sense to you, but here's a path that tends to work well:

1. **Pick your task:** Choose something that naturally benefits from multiple roles. A few starting points: research a topic → write a summary → critique/fact-check it; plan a trip → book/select options → sanity-check the budget and logic; write code → review the code → fix based on feedback. Pick something you can evaluate by eye, so you can actually tell if collaboration helped.
2. **Define your agents:** For each agent, write down its role, what input it expects, what output it should produce, and what tools (if any) it has access to (web search, a calculator, a file, another API). Keep roles narrow and distinct, overlapping roles make coordination messier.

3. **Start with a simple pipeline:** Before reaching for a framework, try wiring the agents together yourself with plain function calls, agent A's output becomes agent B's input. This will teach you what the frameworks are actually solving for.
4. **Introduce structured handoffs:** Don't just pass raw text between agents. Define a clear format for what each agent hands off (e.g. JSON with fields like `findings`, `open_questions`, `confidence`), so information isn't lost or garbled between steps.
5. **Add a feedback loop:** This is where it gets interesting. Let the critic agent send work back to the writer/researcher with specific feedback, and let that agent revise. Decide how many rounds you'll allow and how you'll detect "this is good enough now."
6. **(Optional) Bring in a framework:** Once you understand the flow by hand, try re-implementing it in LangGraph, CrewAI, or AutoGen. Compare: what did the framework give you for free (state management, retries, tool routing) and what did it make more annoying?
7. **Test it on a few real tasks:** Run your system end-to-end on 3-5 different inputs and actually read the outputs. Does the critique catch real issues? Does the final result improve after revision, or does it just go in circles?
8. **Document what you built:** Capture your design decisions, what worked, what didn't, and what you'd do differently with more time.

Creative Freedom

This challenge is intentionally open on how you build it. Some things left entirely up to you:

- The task your agents collaborate on (research, writing, planning, coding, debate, anything with room for division of labor)
- Number of agents (2-3 is the target, but a well-justified 4th is fine)
- Whether agents share one LLM with different prompts, or use different models per role

- Whether you build the orchestration yourself or use a framework (LangGraph, CrewAI, AutoGen, or something else)
- Whether agents can use external tools (search, code execution, APIs) or just reason over text
- How many feedback/revision rounds you allow, and how the system decides when to stop
- CLI, notebook, web app, whatever interface makes sense for showing it off

Use whatever excites you. The only requirement is that there are genuinely multiple agents with distinct roles collaborating, not one prompt wearing different hats in sequence.

Timeline

1 week. Get a simple 2-agent pipeline working end-to-end early, that's your safety net. Then spend remaining time adding the third agent, feedback loops, or a framework migration.

Deliverables

- A GitHub repository containing all your code.
- A README.md inside that repository explaining:
 - What task your agents collaborate on and why you chose it
 - Each agent's role, inputs, outputs, and tools
 - How agents communicate (what gets passed between them, and in what format)
 - How to run the system
 - The tools, models, or frameworks you relied on
 - Any challenges you ran into (coordination issues, infinite loops, agents talking past each other) and what you'd improve with more time
- **A demo showing the full system running end-to-end.** It should show:
 - The task/input you gave the system

- Each agent's output at every step (including at least one feedback/revision round, if your system has one)
- The final result

Team Format

Work solo or in a team, entirely your choice. Team up if you want to tackle a more ambitious pipeline (more agents, tool use, a framework migration), or go solo if you'd rather move at your own pace.

 **Good luck, and have fun with it.** Share your repo link and demo when you're done, excited to see how your agents work together.